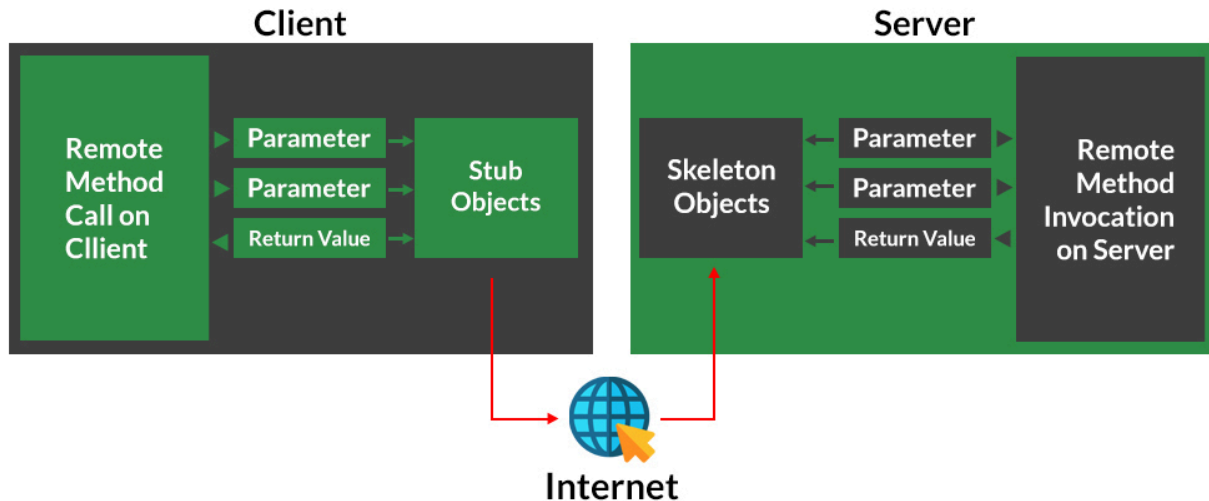


Role of client and server in RMI

Working of RMI



Client-Server Model

The Client-server model is a distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters called clients. In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and delivers the data packets requested back to the client. Clients do not share any of their resources. Examples of the Client-Server Model are Email, World Wide Web, etc.

How Does the Client-Server Model Work?

In this article, we are going to take a dive into the **Client-Server** model and have a look at how the **Internet** works via, web browsers. This article will help

us have a solid WEB foundation and help us easily work with [WEB technologies](#).

- **Client:** When we say the word **Client**, it means to talk of a person or an organization using a particular service. Similarly in the digital world, a **Client** is a computer (**Host**) i.e. capable of receiving information or using a particular service from the service providers (**Servers**).
- **Servers:** Similarly, when we talk about the word **Servers**, It means a person or medium that serves something. Similarly in this digital world, a **Server** is a remote computer that provides information (data) or access to particular services.

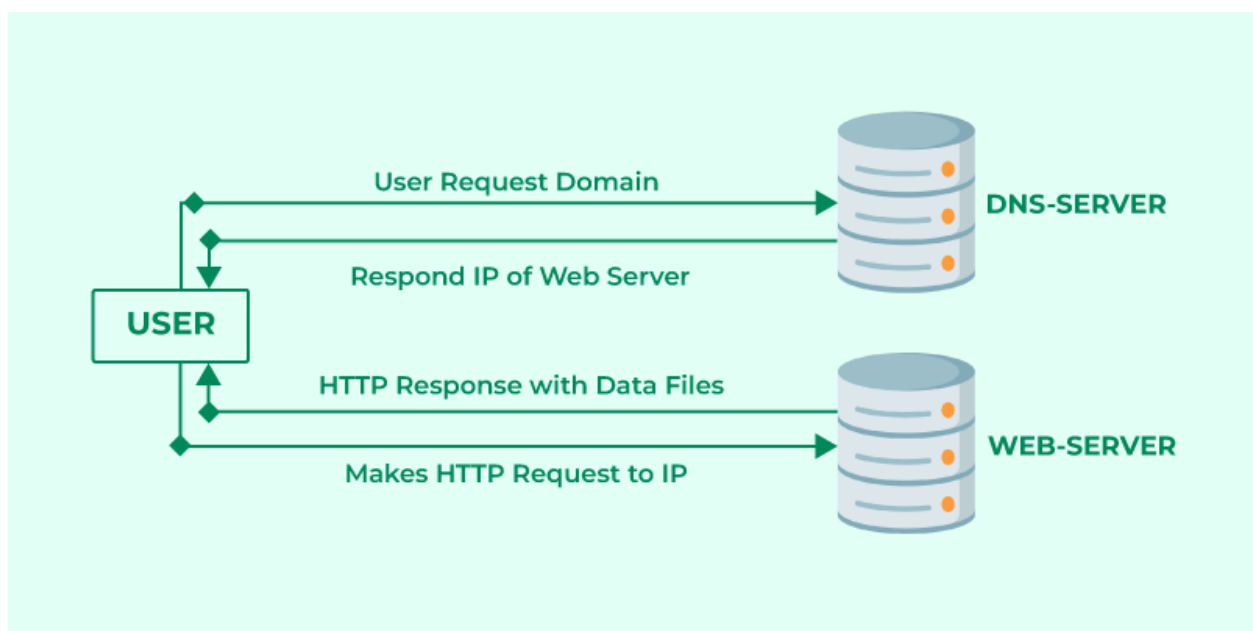
So, it is the **Client** requesting something and the **Server** serving it as long as it is in the database.

How the Browser Interacts With the Servers?

There are a few steps to follow to interact with the servers of a client.

- User enters the **URL**(Uniform Resource Locator) of the website or file. The Browser then requests the [DNS\(DOMAIN NAME SYSTEM\)](#) Server.
- **DNS Server** lookup for the address of the **WEB Server**.
- The **DNS Server** responds with the **IP address** of the **WEB Server**.

- The Browser sends over an **HTTP/HTTPS** request to the **WEB Server's IP** (provided by the **DNS server**).
- The Server sends over the necessary files for the website.
- The Browser then renders the files and the website is displayed. This rendering is done with the help of **DOM** (Document Object Model) interpreter, **CSS** interpreter, and **JS Engine** collectively known as the **JIT** or (Just in Time) Compilers.



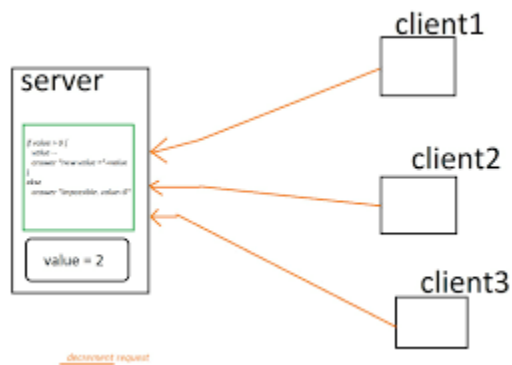
Client Server Request and Response

Advantages of Client-Server Model

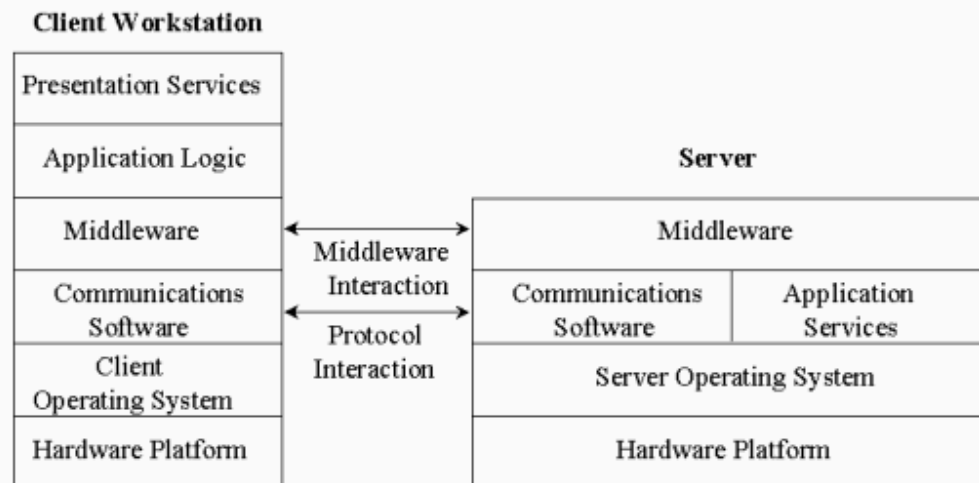
- Centralized system with all data in a single place.
- Cost efficient requires less maintenance cost and Data recovery is possible.
- The capacity of the Client and Servers can be changed separately.

Disadvantages of Client-Server Model

- Clients are prone to viruses, Trojans, and worms if present in the Server or uploaded into the Server.
- Servers are prone to [Denial of Service \(DOS\)](#) attacks.
- Data packets may be spoofed or modified during transmission.
- Phishing or capturing login credentials or other useful information of the user are common and [MITM\(Man in the Middle\)](#) attacks are common.



The Role of Middleware in Client/Server Architecture



RMI (Remote Method Invocation)

The **RMI** (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java. The RMI allows an object to invoke methods on an object running in another JVM.

The RMI provides remote communication between the applications using two objects *stub* and *skeleton*.

Understanding stub and skeleton

RMI uses stub and skeleton object for communication with the remote object.

A **remote object** is an object whose method can be invoked from another JVM. Let's understand the stub and skeleton objects:

stub

The stub is an object, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote object. When the caller invokes method on the stub object, it does the following tasks:

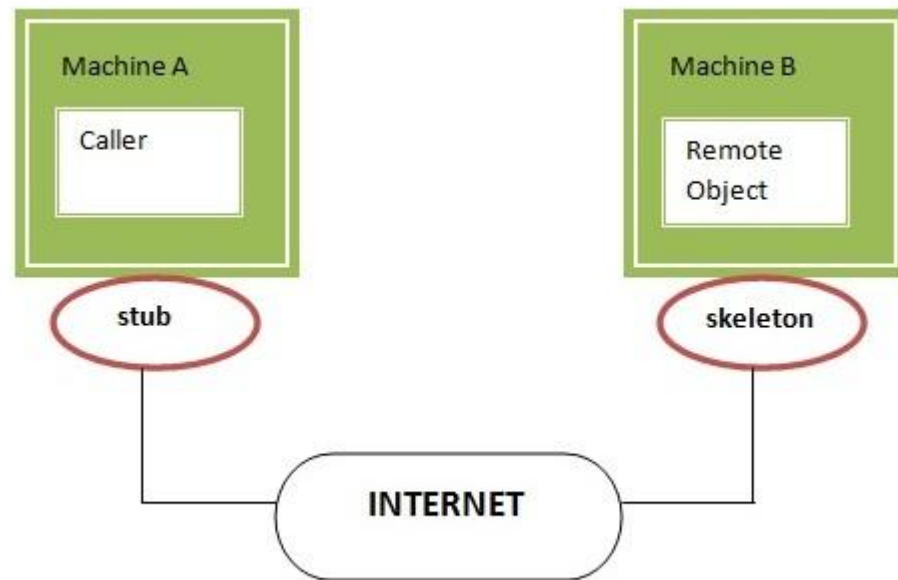
1. It initiates a connection with remote Virtual Machine (JVM),
2. It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
3. It waits for the result
4. It reads (unmarshals) the return value or exception, and
5. It finally, returns the value to the caller.

skeleton

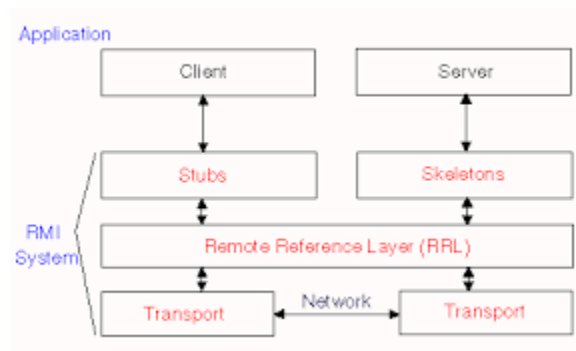
The skeleton is an object, acts as a gateway for the server side object. All the incoming requests are routed through it. When the skeleton receives the incoming request, it does the following tasks:

1. It reads the parameter for the remote method
2. It invokes the method on the actual remote object, and
3. It writes and transmits (marshals) the result to the caller.

In the Java 2 SDK, an stub protocol was introduced that eliminates the need for



skeletons.



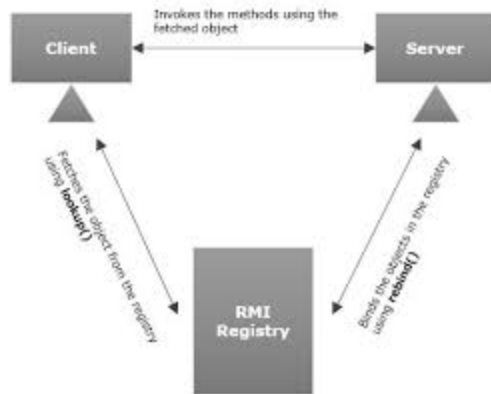
Let us now discuss the components of this architecture.

Transport Layer – This layer connects the client and the server. It manages the existing connection and also sets up new connections.

Stub – A stub is a representation (proxy) of the remote object at client. It resides in the client system; it acts as a gateway for the client program.

Skeleton – This is the object which resides on the server side. stub communicates with this skeleton to pass request to the remote object.

RRL(Remote Reference Layer) – It is the layer which manages the references made by the client to the remote object.



RMI registry is a namespace on which all server objects are placed. Each time the server creates an object, it registers this object with the RMI registry (using `bind()` or `reBind()` methods). These are registered using a unique name known as bind name.

To invoke a remote object, the client needs a reference of that object. At that time, the client fetches the object from the registry using its bind name (using `lookup()` method).

Marshalling and Unmarshalling

Whenever a client invokes a method that accepts parameters on a remote object, the parameters are bundled into a message before being sent over the network. These parameters may be of primitive type or objects. In case of primitive type, the parameters are put together and a header is attached to it. In case the parameters are objects, then they are serialized. This process is known as marshalling.

At the server side, the packed parameters are unbundled and then the required method is invoked. This process is known as unmarshalling.

Understanding requirements for the distributed applications

If any application performs these tasks, it can be distributed application.

.

1. The application need to locate the remote method
2. It need to provide the communication with the remote objects, and
3. The application need to load the class definitions for the objects.

The RMI application have all these features, so it is called the distributed application.

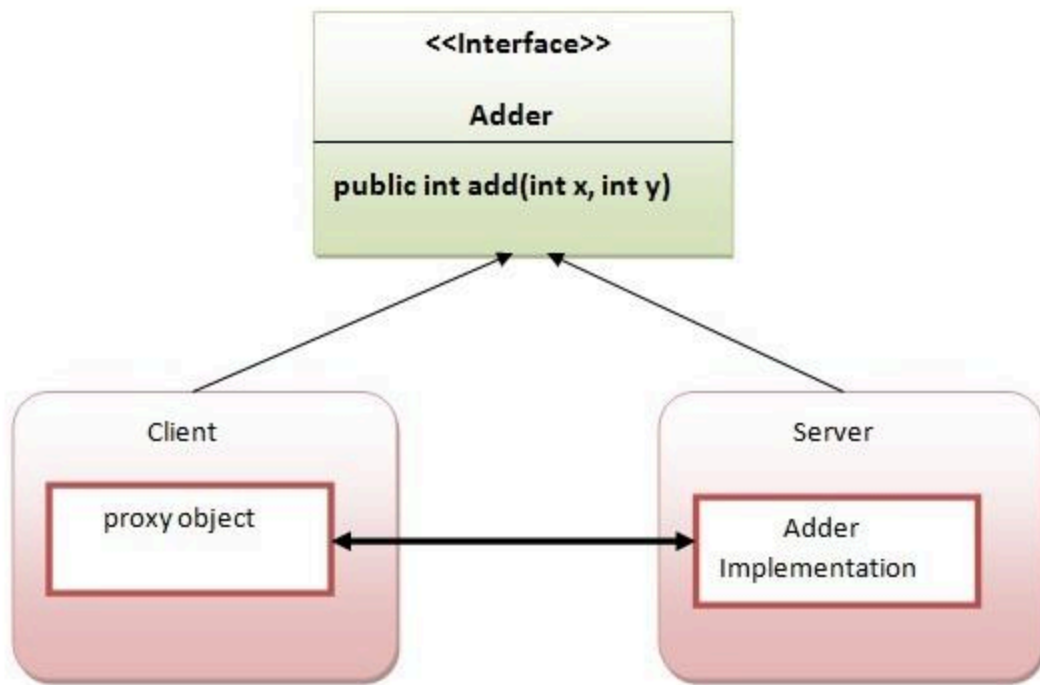
Java RMI Example

The is given the 6 steps to write the RMI program.

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

RMI Example

In this example, we have followed all the 6 steps to create and run the rmi application. The client application need only two files, remote interface and client application. In the rmi application, both client and server interacts with the remote interface. The client application invokes methods on the proxy object, RMI sends the request to the remote JVM. The return value is sent back to the proxy object and then to the client application.



1) create the remote interface

For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface. Here, we are creating a remote interface that extends the Remote interface. There is only one method named add() and it declares RemoteException.

1. **import** java.rmi.*;
2. **public interface** Adder **extends** Remote{
3. **public int** add(**int** x,**int** y)**throws** RemoteException;
4. }

2) Provide the implementation of the remote interface

Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

- Either extend the UnicastRemoteObject class,
- or use the exportObject() method of the UnicastRemoteObject class

In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

1. **import** java.rmi.*;
 2. **import** java.rmi.server.*;
 3. **public class** AdderRemote **extends** UnicastRemoteObject **implements** Adder{
 4. AdderRemote()**throws** RemoteException{
 5. **super**();
 6. }
 7. **public int** add(**int** x,**int** y){**return** x+y;}
 8. }
-

3) create the stub and skeleton objects using the rmic tool.

Next step is to create stub and skeleton objects using the rmi compiler. The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

1. rmic AdderRemote
-

4) Start the registry service by the rmiregistry tool

Now start the registry service by using the rmiregistry tool. If you don't specify the port number, it uses a default port number. In this example, we are using the port number 5000.

1. rmiregistry 5000
-

5) Create and run the server application

Now rmi services need to be hosted in a server process. The Naming class provides methods to get and store the remote object. The Naming class provides 5 methods.

<pre>public static java.rmi.Remote lookup(java.lang.String) throws java.rmi.NotBoundException, java.net.MalformedURLException, java.rmi.RemoteException;</pre>	It returns the reference of the remote object.
<pre>public static void bind(java.lang.String, java.rmi.Remote) throws java.rmi.AlreadyBoundException, java.net.MalformedURLException, java.rmi.RemoteException;</pre>	It binds the remote object with the given name.
<pre>public static void unbind(java.lang.String) throws java.rmi.RemoteException, java.rmi.NotBoundException, java.net.MalformedURLException;</pre>	It destroys the remote object which is bound with the given name.
<pre>public static void rebind(java.lang.String, java.rmi.Remote) throws java.rmi.RemoteException, java.net.MalformedURLException;</pre>	It binds the remote object to the new name.
<pre>public static java.lang.String[] list(java.lang.String) throws java.rmi.RemoteException, java.net.MalformedURLException;</pre>	It returns an array of the names of the remote objects bound in the registry.

In this example, we are binding the remote object by the name sonoo.

1. **import** java.rmi.*;
2. **import** java.rmi.registry.*;

3. **public class** MyServer{
4. **public static void** main(String args[]){
5. **try**{
6. Adder stub=**new** AdderRemote();
7. Naming.rebind("rmi://localhost:5000/sonoo",stub);
8. }**catch**(Exception e){System.out.println(e);}
9. }
- 10.}

6) Create and run the client application

At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object. In this example, we are running the server and client applications, in the same machine so we are using localhost. If you want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

1. **import** java.rmi.*;
2. **public class** MyClient{
3. **public static void** main(String args[]){
4. **try**{
5. Adder stub=(Adder)Naming.lookup("rmi://localhost:5000/sonoo");
6. System.out.println(stub.add(34,4));
7. }**catch**(Exception e){}
8. }
9. }

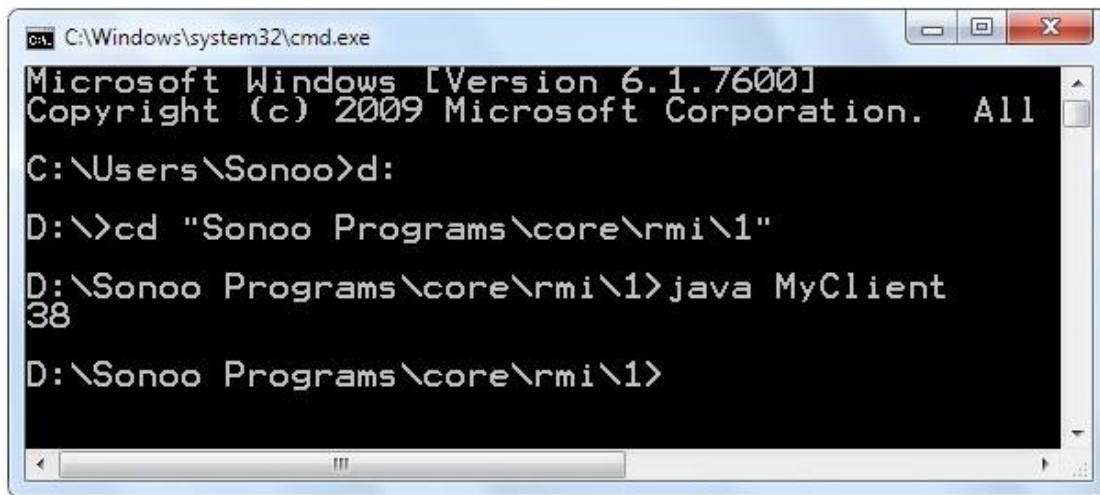
1. For running **this** rmi example,

- 2.
3. 1) compile all the java files
- 4.
5. `javac *.java`
- 6.
7. 2) create stub and skeleton object by `rmic` tool
- 8.
9. `rmic AdderRemote`
- 10.
11. 3) start rmi registry in one command prompt
- 12.
13. `rmiregistry 5000`
- 14.
15. 4) start the server in another command prompt
- 16.
17. `java MyServer`
- 18.
19. 5) start the client application in another command prompt
- 20.
21. `java MyClient`

Output of this RMI example

```
C:\Windows\system32\cmd.exe - rmiregistry 5000
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All
C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>javac *.java
D:\Sonoo Programs\core\rmi\1>rmic AdderRemote
D:\Sonoo Programs\core\rmi\1>rmiregistry 5000
```

```
C:\Windows\system32\cmd.exe - java MyServer
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All
C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>java MyServer
```

A screenshot of a Windows command prompt window. The title bar reads "C:\Windows\system32\cmd.exe". The window contains the following text:

```
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All
C:\Users\Sonoo>d:
D:\>cd "Sonoo Programs\core\rmi\1"
D:\Sonoo Programs\core\rmi\1>java MyClient
38
D:\Sonoo Programs\core\rmi\1>
```

Goals of RMI Following are the goals of RMI -

To minimize the complexity of the application.

To preserve type safety.

Distributed garbage collection.

Minimize the difference between working with local and remote objects.

To write an RMI Java application, you would have to follow the steps given below -

Define the remote interface

Develop the implementation class (remote object)

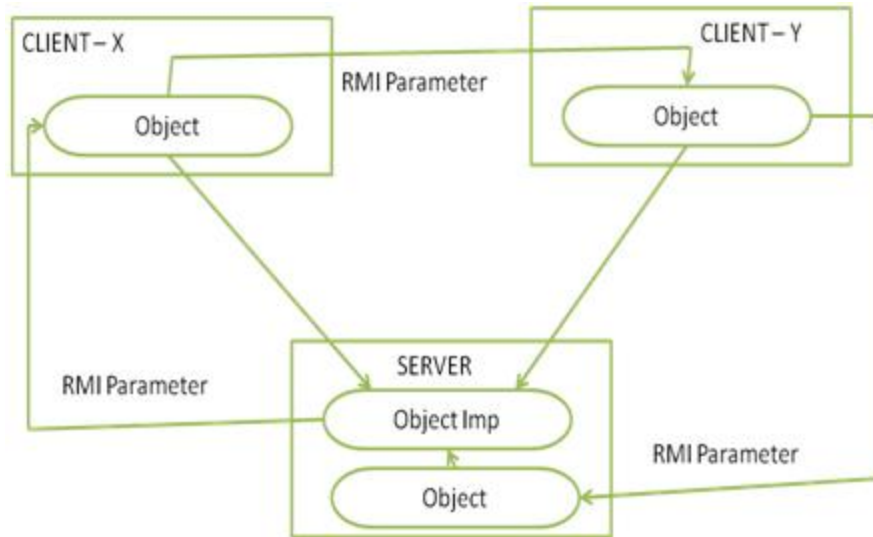
Develop the server program

Develop the client program

Compile the application

Execute the application

Parameter Passing in RMI



Parameters in RMI

RMI supports method calls to remote objects.

1. When these calls involve passing parameters or accepting a return value, it depends on whether the parameters are :

- I. Primitive data types.
- II. Objects.
- III. Remote objects.

1. When the parameter is passed to a method the JVM makes a copy of the value places the copy on the stack and then executes the method.

I. PRIMITIVE PARAMETERS:

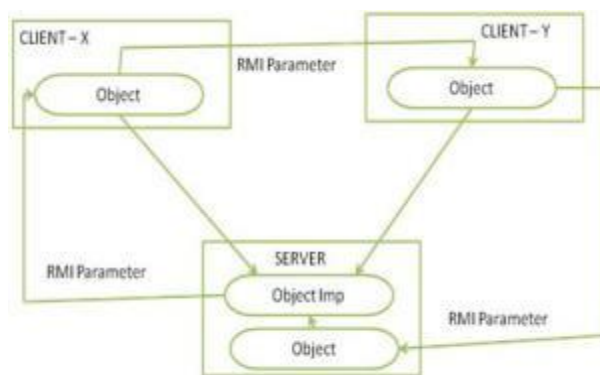
- a) When a primitive data type is passed as a parameter to a remote method, the RMI system passes it by value.
- b) RMI will make a copy of a primitive data type and send it to the remote method. If a method returns a primitive data type, it is also returned to the calling JVM by value.
- c) Values are passed between JVMs in a standard, machine-independent format.
- d) This allows JVMs running on different platforms to communicate with each other reliably.

II. OBJECT PARAMETERS:

- a) When an object is passed to a remote method, the semantics change from the case of the single JVM.

- b) RMI sends the object itself, not its reference, between JVMs.
- c) It is the object that is passed by value, not the reference to the object.
- d) Similarly, when a remote method returns an object, a copy of the whole object is returned to the calling program.
- e) Because different JVMs do not share heap memory, RMI must send the referenced object and all objects it references.
- f) RMI uses a technology called Object Serialization to transform an object into a linear format that can then be sent over the network wire.
- g) Object serialization essentially flattens an object and any objects it references.
- h) Serialized objects can be de-serialized in the memory of the remote JVM and made ready for use by a Java program.

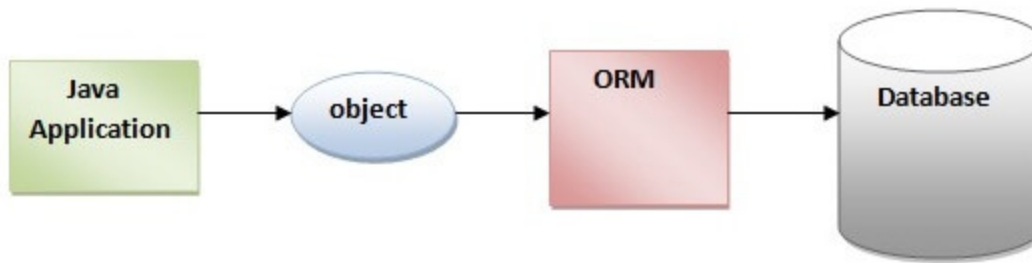
III. REMOTE OBJECT PARAMETERS:



Parameters in RMI

- a) A client program can obtain a remote reference to a remote object, through the RMI registry program.
- b) When a method returns a local reference to an exported remote object, RMI does not return that object.
- c) It substitutes another object (remote proxy for that service), in the return stream.
- d) When a reference is returned to the server, it is not converted into a local reference.
- e) When the object from the server is returned, to the client the proxy object is substituted.

Introduction to hibernate framework



Hibernate is used to overcome the limitations of JDBC like:

1. JDBC code is dependent upon the Database software being used i.e. our persistence logic is dependent, because of using JDBC. Here we are inserting a record into Employee table but our query is Database software-dependent i.e. Here we are using MySQL. But if we change our Database then this query won't work.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
public class InsertDataIntoEmp
{
    public static void main(String[] args)
    {
        final String URL="jdbc:mysql://localhost:3306?user=root&password=root";
        String query="insert into EmployeeDB.Employee values(1,'Bikash Dubey','300000','CSE')";
        try
        {
            Class.forName("com.mysql.jdbc.Driver");
            Connection con=DriverManager.getConnection(URL);
            System.out.println("Connection established");
            Statement st=con.createStatement();
            st.execute(query);
            System.out.println("Data inserted");
            st.close();
            con.close();
        } catch (SQLException | ClassNotFoundException e)
        {
            e.printStackTrace();
        }
    }
}
```

2. If working with JDBC, changing of Database in middle of the project is very costly.
3. JDBC code is not portable code across the multiple database software.

4. In JDBC, Exception handling is mandatory. Here We can see that we are handling lots of Exception for connection.

```
import java.io.IOException;
import java.sql.*;
public class Demo
{
    public static void main(String[] args) throws IOException, SQLException
    {
        try {
            /*loading & register of the driver class*/
            Class.forName("com.mysql.jdbc.Driver");
            Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306","root", "root");
            System.out.println("Driver class loaded successfully");
        } catch (ClassNotFoundException e)
        {
            System.out.println("Driver Class not loaded");
        }
    }
}
```

5. While working with JDBC, There is no support Object-level relationship.
6. In JDBC, there occurs a Boilerplate problem i.e. For each and every project we have to write the below code. That increases the code length and reduce the readability.

```
try
{
    Class.forName("com.mysql.jdbc.Driver");
    Connection con=DriverManager.getConnection("jdbc:mysql://localhost:3306?user=root&password=root");
    System.out.println("Connection established");
    con.close();
} catch (SQLException | ClassNotFoundException e)
{
    e.printStackTrace();
}
```

To overcome the above problems we use ORM tool i.e. nothing but Hibernate framework. By using Hibernate we can avoid all the above problems and we can enjoy some additional set of functionalities.

About Hibernate Framework

Hibernate is a framework which provides some **abstraction layer**, meaning that the programmer does not have to worry about the implementations, Hibernate does the implementations for you internally like **Establishing a**

connection with the database, writing query to perform CRUD operations etc.

It is a **java framework** which is used to develop persistence logic. Persistence logic means to store and process the data for long use. More precisely Hibernate is an open-source, non-invasive, light-weight java ORM(Object-relational mapping) framework to develop objects which are independent of the database software and make independent persistence logic in all JAVA, JEE.

Framework means it is special install-able software that provides an abstraction layer on one or more technologies like JDBC, Servlet, etc to simplify or reduce the complexity for the development process.

Open Source means:

- Hibernate framework is available for everyone without any cost.
- The source code of Hibernate is also available on the Internet and we can also modify the code.

Light-weight means:

- Hibernate is less in size means the installation package is not big in size.
- Hibernate does not require any heavy container for execution.
- It does not require POJO and POJI model programming.
- Hibernate can be used alone or we can use Hibernate with other java technology and framework.

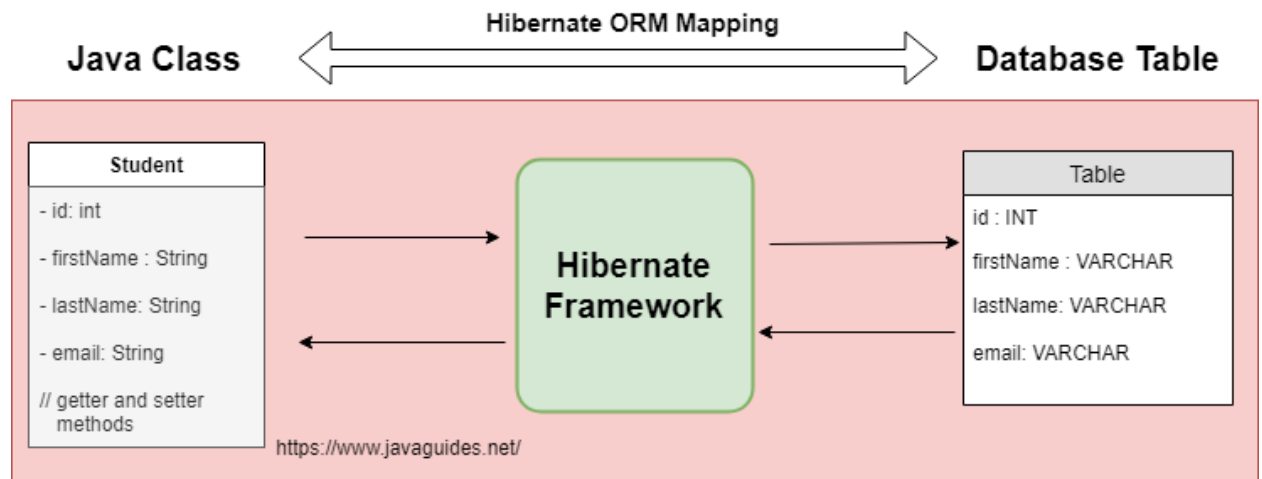
Non-invasive means:

- The classes of Hibernate application development are loosely coupled classes with respect to Hibernate API i.e. Hibernate class need not implement hibernate API interfaces and need not extend from Hibernate API classes.

Functionalities supported by Hibernate framework

- Hibernate framework support **Auto DDL** operations. In JDBC manually we have to create table and declare the data-type for each and every column. But Hibernate can do **DDL operations** for you internally like creation of table,drop a table,alter a table etc.
- Hibernate supports **Auto Primary key generation**. It means in JDBC we have to manually set a primary key for a table. But Hibernate can this task for you.
- Hibernate framework is independent of Database because it supports **HQL (Hibernate Query Language)** which is not specific to any database, whereas JDBC is database dependent.
- In Hibernate, **Exception Handling is not mandatory**, whereas In JDBC exception handling is mandatory.
- Hibernate supports **Cache Memory** whereas JDBC does not support cache memory.

- Hibernate is a **ORM tool** means it support Object relational mapping. Whereas JDBC is not object oriented moreover we are dealing with values means primitive data. In hibernate each record is represented as a Object but in JDBC each record is nothing but a data which is nothing but primitive values.



Hibernate Architecture

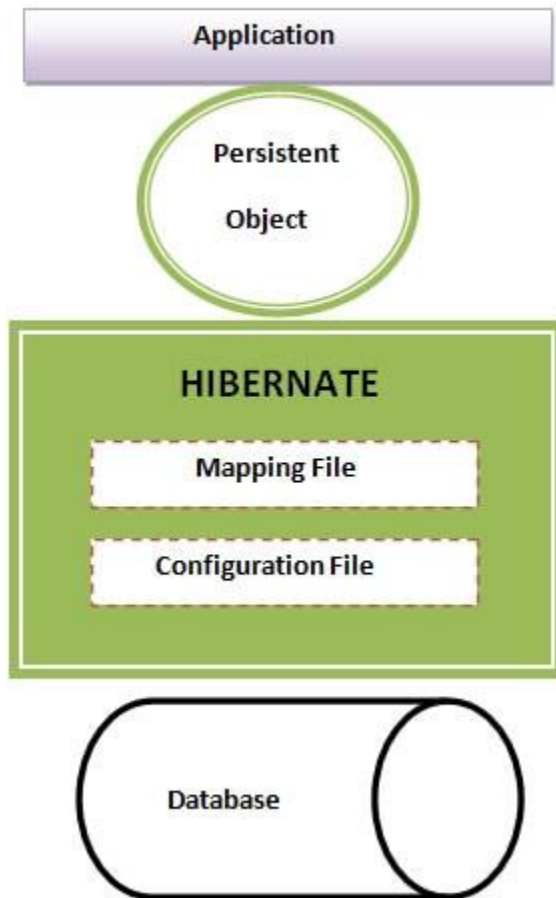
The Hibernate architecture includes many objects such as persistent object, session factory, transaction factory, connection factory, session, transaction etc.

The Hibernate architecture is categorized in four layers.

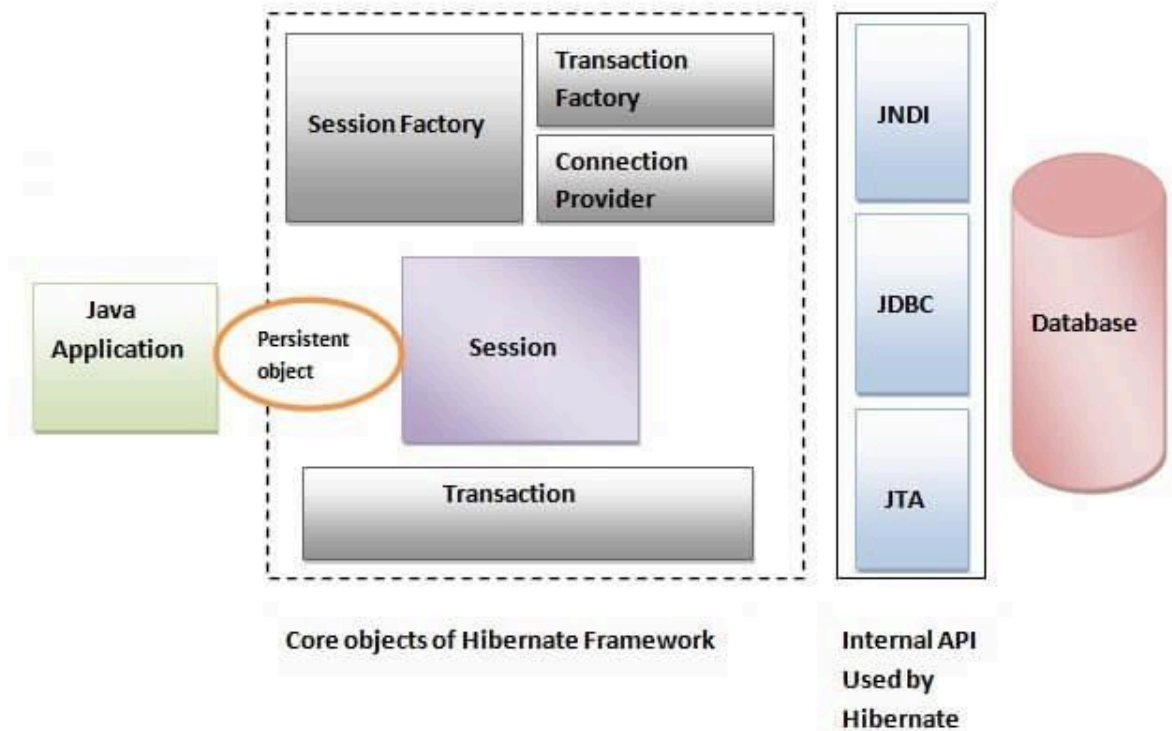
- Java application layer
- Hibernate framework layer
- Backhand api layer

- Database layer

Let's see the diagram of hibernate architecture:



This is the high level architecture of Hibernate with mapping file and configuration file.



Hibernate framework uses many objects such as session factory, session, transaction etc. alongwith existing Java API such as JDBC (Java Database Connectivity), JTA (Java Transaction API) and JNDI (Java Naming Directory Interface).

Elements of Hibernate Architecture

Configuration:

- Configuration is a class which is present in `org.hibernate.cfg` package. It activates Hibernate framework. It reads both configuration file and mapping files.

It activate Hibernate Framework

```
Configuration cfg=new Configuration();
```

It read both cfg file and mapping files

```
cfg.configure();
```

- It checks whether the config file is syntactically correct or not.
- If the config file is not valid then it will throw an exception. If it is valid then it creates a meta-data in memory and returns the meta-data to object to represent the config file.

SessionFactory:

- SessionFactory is an Interface which is present in org.hibernate package and it is used to create Session Object.
- It is immutable and thread-safe in nature.

buildSessionFactory() method gathers the meta-data which is in the cfg Object.

From cfg object it takes the JDBC information and create a JDBC Connection.

```
SessionFactory factory=cfg.buildSessionFactory();
```

Session:

- Session is an interface which is present in org.hibernate package. Session object is created based upon SessionFactory object i.e. factory.

- It opens the Connection/Session with Database software through Hibernate Framework.
- It is a light-weight object and it is not thread-safe.
- Session object is used to perform CRUD operations.

```
Session session=factory.buildSession();
```

Transaction:

- Transaction object is used whenever we perform any operation and based upon that operation there is some change in database.
- Transaction object is used to give the instruction to the database to make the changes that happen because of operation as a permanent by using commit() method.

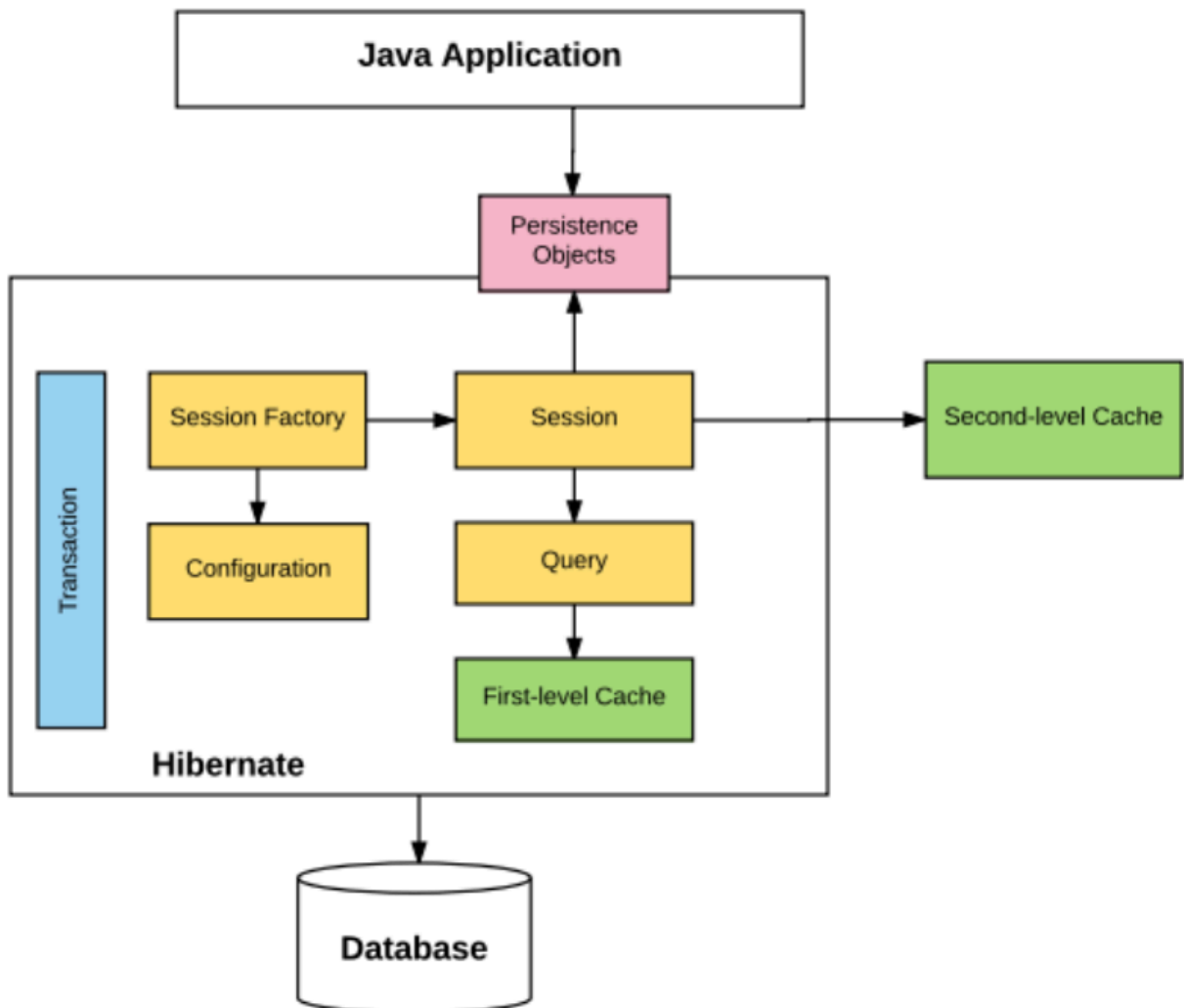
```
Transaction tx=session.beginTransaction();  
tx.commit();
```

Query:

- Query is an interface that present inside org.hibernate package.
- A Query instance is obtained by calling Session.createQuery().
- This interface exposes some extra functionality beyond that provided by Session.iterate() and Session.find():
 1. A particular page of the result set may be selected by calling setMaxResults(), setFirstResult().
 2. Named query parameters may be used.

Criteria:

- Criteria is a simplified API for retrieving entities by composing Criterion objects.
- The Session is a factory for Criteria. Criterion instances are usually obtained via the factory methods on Restrictions.



Persistence

- Persistence simply means that we would like our application's data to outlive the applications process. In Java terms, we would like the state of (some of) our objects to live beyond the scope of the JVM so that the same state is available later.

Internal API used by Hibernate

1. JDBC (Java Database Connectivity)
2. JTA (Java Transaction API)
3. JNDI (Java Naming Directory Interface)

Objects/Elements Of Hibernate Architecture

1. Configuration Object
 - The Configuration object is the first Hibernate object you create in any Hibernate application.
 - It is usually created only once during application initialization.
 - The Configuration object provides two keys components:
 1. Database Connection:
 - This is handled through one or more configuration files supported by Hibernate. These files are hibernate.properties and hibernate.cfg.xml.
 2. Class Mapping Setup:
 - This component creates the connection between the Java classes and database tables.
2. SessionFactory Object
 - The SessionFactory is a thread safe object and used by all the threads of an application.
 - Configuration object is used to create a SessionFactory object which in turn configures Hibernate for the application.
 - You would need one SessionFactory object per database using a separate configuration file.
 - So, if you are using multiple databases, then you would have to create multiple SessionFactory objects.
3. Session Object
 - A Session is used to get a physical connection with a database.
 - The Session object is lightweight and designed to be instantiated each time an interaction is needed with the database.
 - The session objects should not be kept open for a long time because they are not usually thread safe and they should be created and destroyed as needed.
4. Transaction Object
 - A Transaction represents a unit of work with the database and most of the RDBMS supports transaction functionality.
 - Transactions in Hibernate are handled by an underlying transaction manager and transaction (from JDBC or JTA).
5. Query Object
 - Query objects use SQL or Hibernate Query Language (HQL) string to retrieve data from the database and create objects.

- A Query instance is used to bind query parameters, limit the number of results returned by the query, and finally to execute the query.
6. Criteria Object
- Criteria objects are used to create and execute object oriented criteria queries to retrieve objects.